

Parcial 2 - LivreStream - Segunda parte - Resolución

Punto 1

Diagrama de clases

Pseudocódigo

Canal

Transmisión

Moderación

Vigencia

Ejemplo de uso

Notas

Punto 2

Diagrama de clases

Pseudocódigo

Notas

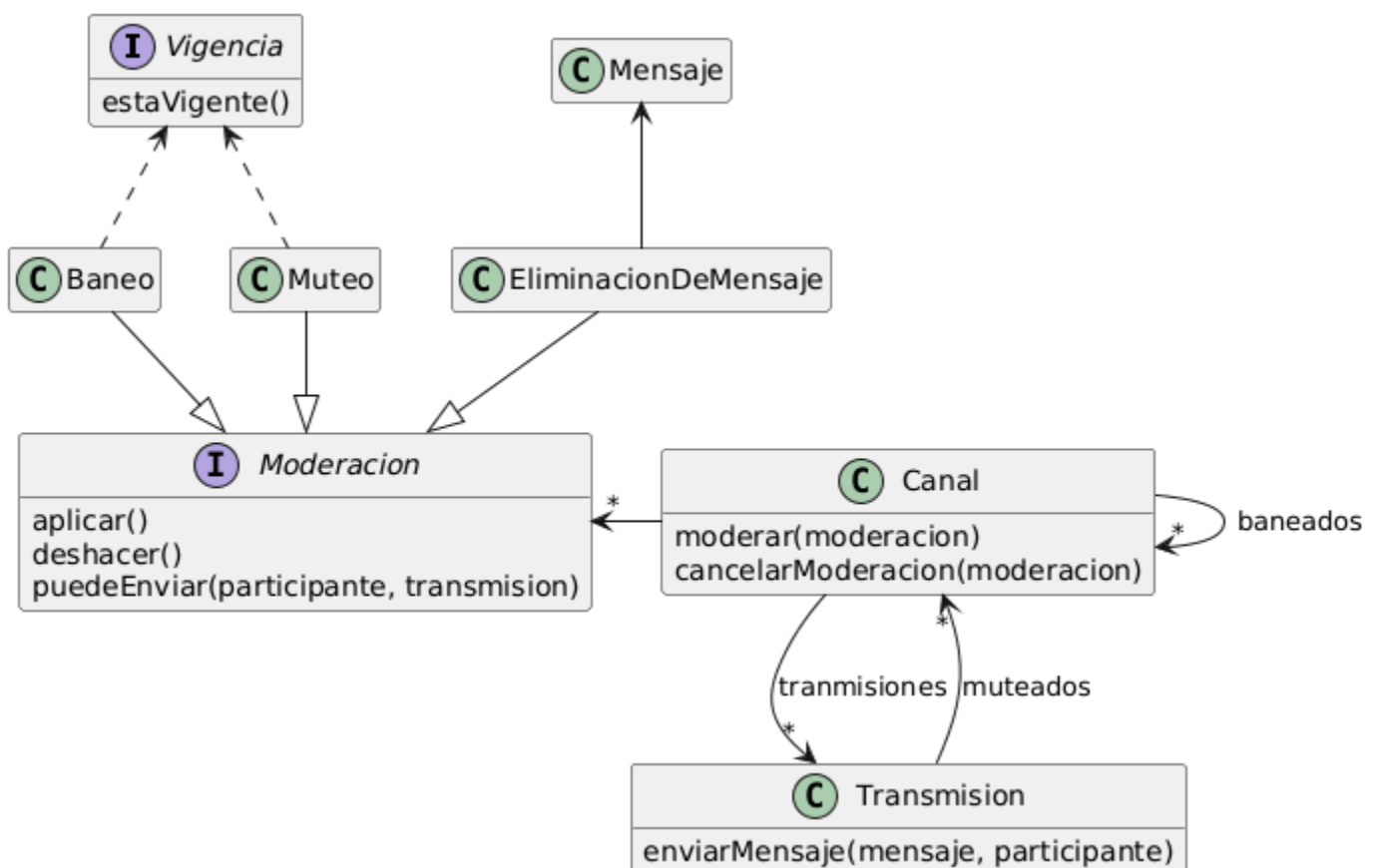
Punto 3

Punto 4

Punto 1

Como persona administradora de un canal, deseo ejecutar y deshacer acciones de moderación sobre un participante de un stream.

Diagrama de clases



Pseudocódigo

Canal

```
class Canal {
  modear(Moderacion moderacion)
    this.moderaciones.add(moderacion)
    moderacion.aplicar()
  }

  cancelarModeracion(Moderacion moderacion) {
    // Esto es opcional, porque no se aclara si
    // se debe mantener registro histórico de las moderaciones
    // aplicadas
    // this.moderaciones.remove(moderacion)
    moderacion.deshacer()
  }

  banear(Canal participante) {
    this.participantesBaneados.add(participante)
  }

  desbanear(Canal participante) {
    this.participantesBaneados.remove(participante)
  }

  tieneBaneo(Canal participante) {
    return this.participantesBaneados.includes(participante)
  }

  puedeEnviarMensaje(Canal participante, Transmision transmision) {
    return this.moderaciones.all(it => it.puedeEnviarMensaje(
      participante, transmision))
  }
}
```

Transmisión

```
class Transmision {
    mutear(Canal participante) {
        this.participantesMuteados.add(participante)
    }

    desmutear(Canal participante) {
        this.participantesMuteados.remove(participante)
    }

    tieneBaneo(Canal participante) {
        // nota: acá tieneBaneo significa que se le ha aplicado
        // un baneo, aunque no necesariamente este está vigente
        return this.canal.tieneBaneo(participante)
    }

    tieneMuteo(Canal participante) {
        // idem comentario anterior
        return this.participantesMuteados.includes(participante)
    }

    enviarMensaje(Mensaje mensaje, Canal participante) {
        if (this.canal.puedeEnviarMensaje(participante, this)) {
            this.chat.enviarMensaje(mensaje)
        } else {
            throw new NoSePuedeEnviarMensajeException(...)
        }
    }
}
```

Moderación

```
interface Moderacion {
    aplicar()
    deshacer()
    puedeEnviarMensaje(participante, transmision): boolean
}

class EliminacionDeMensaje {
    // si el mensaje conoce a su chat, se puede obviar this.chat
    aplicar() { this.chat.eliminarMensaje(this.mensaje) }
    deshacer() { this.chat.publicarMensaje(this.mensaje) }
    puedeEnviarMensaje(participante, transmision) { return true }
}

class Muteo {
    aplicar() { this.transmision.mutear(this.participante) }
    deshacer() { this.transmision.desmutear(this.participante) }
    puedeEnviarMensaje(participante, transmision) {
        // estamos asumiendo que hay un solo muteo para un participante por vez
        // y que se llama desde el lugar correcto
        // esto no necesariamente es cierto, se podría complejizar un poco más esta lógica
        return transmision.tieneMuteo(participante) && this.vigencia.estaVigente()
    }
}

class Baneo {
    aplicar() { this.canal.banear(this.participante) }
    deshacer() { this.canal.desbanear(this.participante) }
    puedeEnviarMensaje(participante, transmision) {
        // idem anterior
        return transmision.tieneBaneo(participante) && this.vigencia.estaVigente()
    }
}
```


Vigencia

```
interface Vigencia {  
    estaVigente()  
}  
  
class SinExpiracion {  
    estaVigente() { return true }  
}  
  
class ConExpiracion {  
    estaVigente() { return this.expiration >= LocalDate.now() }  
}
```

Ejemplo de uso

```
// =====
// Como persona administradora de un canal...
// =====

Canal miCanal = // ...

// =====
// ... deseo ejecutar...
// =====

Mensaje unMensajeInapropiado = // ...
Moderacion eliminarMensajeInapropiado = new
EliminacionDeMensaje(unMensajeInapropiado)
miCanal.moderar(eliminarMensajeInapropiado)

Canal feli = //...
Moderacion banearAFeli = new Baneo(
    miCanal, feli,
    new SinExpiracion() // se podría reutilizar SinExpiracion, pero no es importante
    para el parcial
)
miCanal.moderar(banearAFeli)

// =====
//...y deshacer acciones de moderación
// sobre un participante de un stream.
// =====

miCanal.cancelarModeracion(eliminarMensajeInapropiado)

// =====
// Además es fácil administrar las
// moderaciones...
// =====

miCanal.moderaciones

// =====
// ...y las mismas siempre tienen
// efecto al enviar los mensajes
// =====

Canal dani = ...
Transmision unaTransmisionCualquiera = miCanal.transmisiones.atRandom()
unaTransmisionCualquiera.enviarMensaje(new Mensaje(...), dani)
```


Notas

La lista de moderaciones del canal sirve para llevar registro de todas las moderaciones que se han disparado desde este canal a otros canales participantes. Si bien también se podría haber pensado al revés (que un participante tenga la lista de las moderaciones que le han aplicado) eso dificultaría el requerimiento de, como persona administradora de un canal, listar todas las moderaciones para gestionarlas.

Esta lista podría ser tratada como la lista de las moderaciones vigentes, como la lista que también contiene las moderaciones no deshechas o como la lista histórica de moderaciones. En esta solución se ha optado por la tercera opción pero el diseño no debería variar demasiado en los otros escenarios.

Por otro lado, la idea de la interfaz *Vigencia* es una forma sencilla de resolver la lógica común de moderaciones que pueden tener una expiración asociada. Dado que la lógica común entre *Baneos* y *Muteos* es mínima (`&& this.vigencia.estaVigente()`) (y al mismo tiempo no se comparte con las eliminación de mensajes) no se ha introducido ninguna clase abstracta común, aunque bien se podría haberlo hecho mediante, por ejemplo, un template method.

Cabe destacar también que el *AnalizadorDeTexto* es la interfaz original, ya provista, dado que es un componente interno: aún si este tuviera dependencias externas, asumiremos que su gestión ya está resuelta y que al ser una interfaz, en última instancia, podríamos *mockearla* sin problemas.

Por último, alternativamente se podría resolver este requerimiento sin listas accesorias de muteos y baneos. De esa forma, no sería necesario hacer nada participar en aplicar y deshacer en *Muteo* y *Baneo*, y sus *puedeEnviarMensaje* sólo necesitarían evaluar la vigencia y la correspondencia con el participante y transmisión:

```
class EliminacionDeMensaje implements Moderacion {
    aplicar() { this.chat.eliminarMensaje(this.mensaje) }
    deshacer() { this.chat.publicarMensaje(this.mensaje) }
    puedeEnviarMensaje(participante, transmision) { return true }
}

abstract class Silencio implements Moderacion {
    aplicar() { }
    deshacer() { }
    puedeEnviarMensaje(participante, transmision) {
        return this.participante == participante && this.vigencia.estaVigente() &&
this.mismaTransmision(transmision)
    }
}

class Muteo extends Silencio {
    mismaTransmision(transmision) {
        return this.transmision == transmision
    }
}

class Baneo extends Silencio {
    mismaTransmision(transmision) {
        return this.canal == transmision.canal
    }
}
```

```

// o hacer un includes / contains:
// return this.canal.transmite(transmision)
}
}

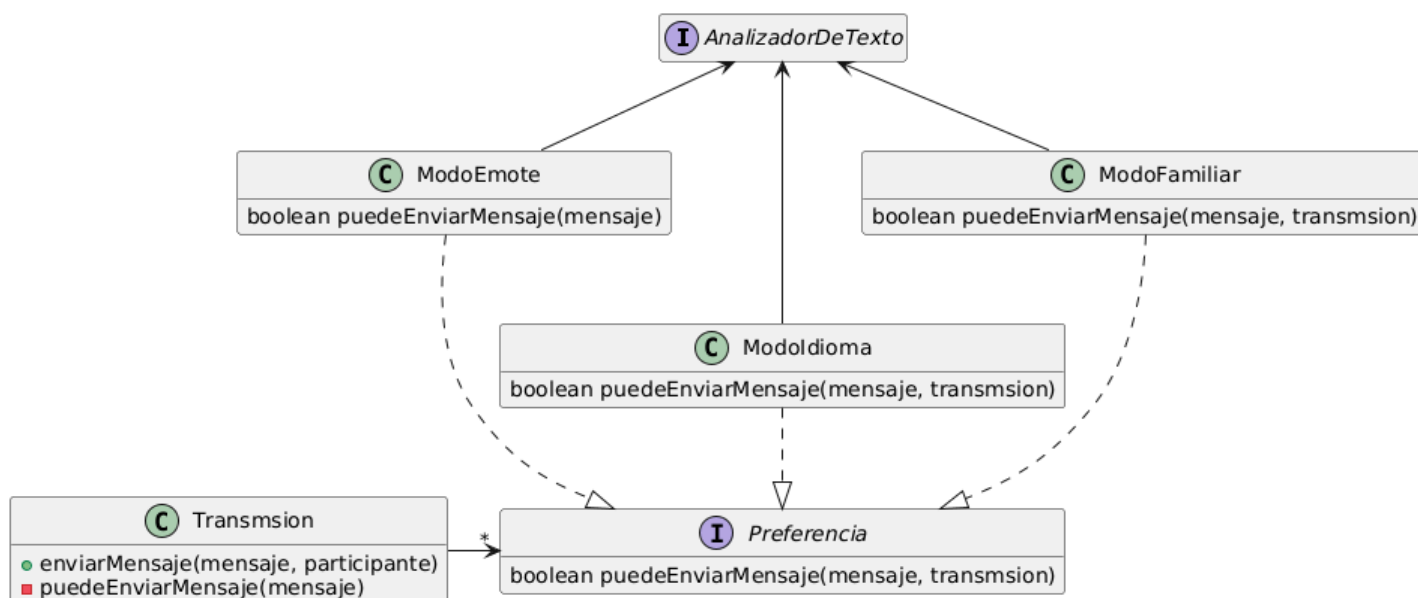
```

Esta solución es, de hecho, más simple y cumple los requerimientos. Además, acá se justifica aún más aplicar una estructura de método plantilla. Como contra, responder las preguntas individuales de quienes son los participantes baneados o muteados en canales y transmisiones se hace más difícil y puede llegar a ser más difícil gestionar en el futuro listados históricos.

Punto 2

Como persona administradora de un canal, deseo cargar mis preferencias en la transmisión en curso y que tengan efecto sobre todos los mensajes que se envían al chat.

Diagrama de clases



Pseudocódigo

```
interface Preferencia {
    boolean puedeEnviarMensaje(Mensaje mensaje, Transmsion transmsion)
}

class ModoEmote implements Preferencia {
    boolean puedeEnviarMensaje(Mensaje mensaje) {
        return analizadorDeTexto.soloContieneEmojis(mensaje)
    }
}

class ModoIdioma implements Preferencia {
    boolean puedeEnviarMensaje(Mensaje mensaje, Transmsion transmsion) {
        return analizadorDeTexto.obtenerIdioma(mensaje) === transmsion.idioma
    }
}

class ModoFamiliar implements Preferencia {
    // análogo a las preferencias anteriores
}

class Transmsion {
    enviarMensaje(Mensaje mensaje, Canal participante) {
        if (
            this.canal.puedeEnviarMensaje(participante, this)
            && this.puedeEnviarMensaje(mensaje)) {
            // ... idem punto anterior...
        } else {
            // ... idem punto anterior...
        }
    }

    puedeEnviarMensaje(Mensaje mensaje) {
        return this.preferencias.all(it => it.puedeEnviarMensaje(mensaje, this))
    }
}
```

Notas

Notar que la interfaz *Preferencia* es bastante parecida a la interfaz de Moderación, pero por un lado expone menos mensajes y por el otro la firma de *puedeEnviarMensaje* es diferente. Dependiendo cómo evolucione el sistema podría eventualmente llegar a surgir una idea común y reutilizable, pero bajo los requerimientos actuales esta similitud es más bien accidental: las *Moderaciones* operan sobre una participante, mientras que las *Preferencias*, sobre un mensaje. De hecho, bien se podrían haber llamado a ambos mensajes de forma diferente.

Observar también que en el *Modoldioma* que estamos utilizando el idioma de la transmisión en lugar de un idioma específico del modo. De esta forma, si el idioma de la transmisión cambia, el modo sigue funcionando correctamente.

Punto 3

Como *persona participante de un chat*, deseo que cada vez que envíe un mensaje, todas las demás personas participantes lo reciban en sus computadoras.

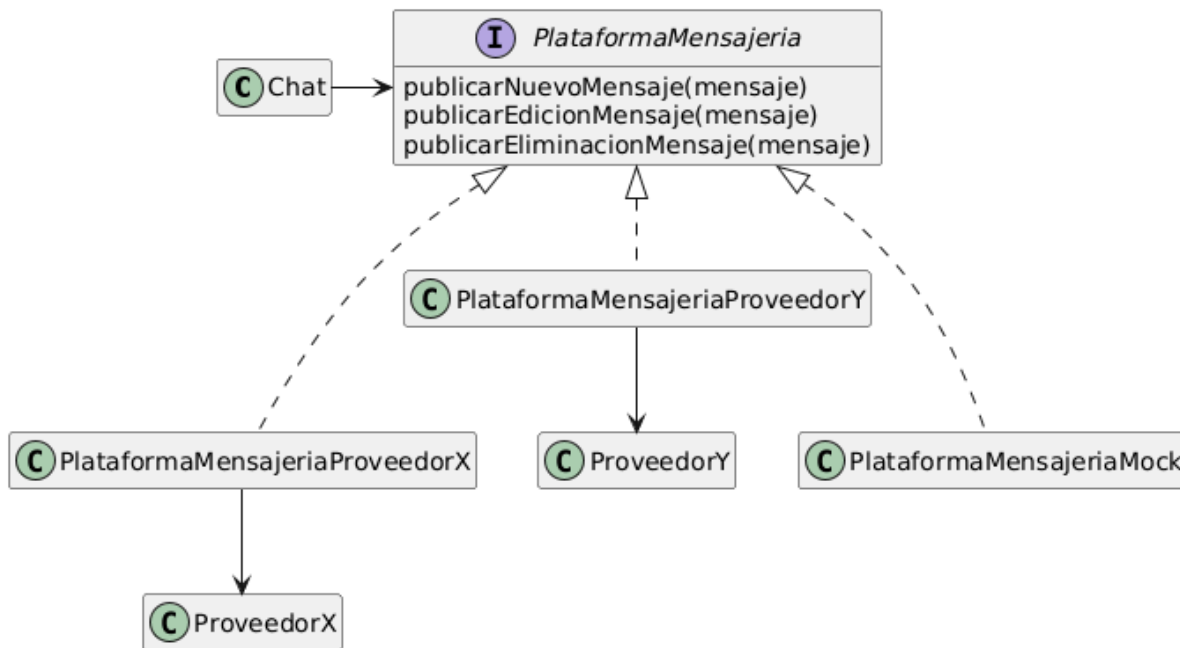
Debemos implementar una estructura estilo adapter:

```
class Chat {
    // inyectado por constructor u obtenido a través de un singleton
    // o service locator
    private plataformaMensajeria: PlataformaMensajeria

    void enviarMensaje(Mensaje mensaje) {
        this.mensajes.add(mensaje)
        this.plataformaMensajeria.publicarNuevoMensaje(mensaje)
    }

    void editarMensaje(Mensaje mensaje) {
        this.plataformaMensajeria.publicarActualizacionMensaje(mensaje)
    }

    void eliminarMensaje(Mensaje mensaje) {
        this.mensaje.remove(mensaje)
        this.plataformaMensajeria.publicarEliminacionMensaje(mensaje)
    }
}
```



Es importante destacar que en esta solución la *PlataformaMensajeria* es una interfaz propia, lo mismo cada implementación de la misma, y sólo las clases *ProveedorX*, *ProveedorY*, etc representan componentes externos.

Punto 4

Como parte del comité de cuidados de LiveStream deseo que las transmisiones terminen siempre a las 2 horas.

Para resolver este requerimiento deberemos algún mecanismo de planificación externa que permita, periódicamente, verificar qué transmisiones están aún activas y cerrarlas en caso de que sea necesario.

La opción más sencilla para esto es utilizar la herramienta nativa de calendarización de los sistemas operativos tipo UNIX, crontab, configurando a la misma para que, cada una cantidad de tiempo significativamente menor a 2 horas, ejecute un JAR dedicado a ésta tarea. Por ejemplo:

```
*/5 * * * * java -jar /home/livestream/terminadorDeTransmisiones.jar
```

Este JAR contendrá el código de nuestra aplicación, sus dependencias y el siguiente código:

```

class TerminadorDeTransmisiones {

    public static void main(string[] args) {
        RepositorioTransmisiones
            .INSTANCE
            .pendientesDeCierre()
            .forEach(it => it.cerrar())
    }
}

class RepositorioTransmisiones {
    List<Transmisiones> pendientesDeCierre() {
        this.transmisiones.filter(it => it.estaPendienteDeCierre())
    }
}

class Transmision {
    boolean estaPendienteDeCierre() {
        return LocalDate.now() >= this.fechaDeInicio.plus(2, TimeUnit.HOURS)
    }

    void cerrar() {
        ....
    }
}

```

